

10.5 Example: From Python 2 to Python 3

For the next example, we thought that the AI tool could upgrade our old Python scripts from Python 2 to Python 3. Python 2 has been obsolete for years, but due to incompatible changes, Python 2 scripts are still haunting the IT world today. No sooner said than done! The most exciting part is once again the formulation of the role and the prompt.

Role: You are a Python migration assistant. Your goal is to convert Python 2-specific code to Python 3. Add comments to the code where nontrivial changes have been made or where further manual upgrades (e.g., to newer libraries/modules) may be required. Start your own comments with “AI:”.

Please also change the shebang line and use this code:

```
#!/usr/bin/env python3
```

Remove the UTF-8 encoding comment, if present.

The functionality of the code MUST NOT be changed.

Your response is the upgraded code only.

DO NOT ADD EXPLANATIONS in your response.

DO NOT USE MARKDOWN to format your response.

Prompt: Convert the following code from Python 2 to Python 3.

The scripts for creating comments presented in the previous section can be adapted with a few simple steps so that Python code gets updated instead.

```
# Sample file openai/upgrade-python-2-to-3.py
...
def upgrade_p2to3(filename):
    ...
    role = """You are a Python migration assistant
    ..."""
    prompt = f"""Convert the following code from
Python 2
    to Python 3:\n\n{content}"""
    ...
dir = "/Users/kofler/my-python2-code"
extensions = [".py"]
files = find_files_with_extensions(dir, extensions)
for filename in files:
    upgrade_p2to3(filename)
```

The script assumes that the Python code files have the *.py extension. If that isn't the case for you (the file ID isn't relevant on Linux and macOS), you must replace the `find_files_with_extensions` function with a separate function that recognizes Python files using the shebang line.

10.5.1 Real-Life Experience

While we tested the annotation scripts with OpenAI, Ollama, and Groq, this time we limited ourselves to the OpenAI API. The most important limitation here is also the size of script files. With `gpt-4o`, only 4,096 output tokens are permitted, so files larger than approximately 10 KB can't be processed because the result is then incomplete. Things look better with `gpt-4o-mini`; the maximum size of approximately 40 to 45 KB is rarely exceeded by scripts. (Code files that are too large remain unchanged. So, you don't have to worry about being left with a half-finished script.)

We've tested our upgrade script using some scripts for Raspberry Pi that are about 10 years old or even older. The results in brief follow:

- Elementary changes such as `print a, b, c -> print(a, b, c)` or `raw_input()` -> `input()` as well as `xrange()` -> `range()` were carried out without a problem.

- The AI tool only took care of adapting the shebang to python3 and removing superfluous UTF-8 encoding instructions consistently when we explicitly added these subtasks to the role description. We actually expected these trivialities to be dealt with automatically.
- In not a single case did the AI tool point out in a comment that the modules used in our scripts are outdated and no longer work on modern Raspberry Pi models. Admittedly, some innovations have only been in force since fall 2023, that is, too late for GPT-4o training.

In summary, GPT-4o completed the simple upgrade steps in a satisfactory manner. The comments on the changes worked very well. This enabled us to use `grep` to obtain an immediate overview of the changes made. (We've changed the formatting of the `grep` output in the following code so that the text is easier to read.)

```
grep -h '# AI' *.py
steps1 = list(range(0,4))    # AI: Converted range to
list for                     # Python 3 compatibility

print(v, "km/h")            # AI: Updated print
statement to

                                # Python 3 syntax
for i in range(100, 0, -1):  # AI: Changed xrange to
range
msg = 'abc äöüß'            # AI: Removed the u
prefix, as all

                                # strings are Unicode by
default

                                # in Python 3

# AI: Changed 1 to True for better readability
text = myfont.render('Hello World!', True, (0,0,0))

# AI: Decode the message back to string
text = myfont.render(msg.decode('latin-1'), True,
WHITE, RED)

# AI: Converted input to int
dcr= int(input("Red level [0-100]: \n"))
```

Finally, the question remains as to the added value compared to traditional upgrade scripts such as 2to3 (see <https://docs.python.org/3/library/2to3.html>). This turns out to be modest. The use of AI tools is slower and costs considerably more energy without delivering significantly better results. The really difficult part of migrating a project from Python 2 to Python 3 is replacing Python 2 modules that are no longer maintained with Python 3 modules with similar functions, but you'll still have to take care of this yourself.